

Complete RAG Crash Course With LangChain in 2 Hours — Transcript

Source video: Krish Naik — https://www.youtube.com/watch?v=o126p1QN_RI (Cleaned from the timestamped transcript PDF you provided — line breaks reflowed into sentences, timestamps kept at natural breaks for reference.)

[0:00] Hello all, my name is Krishna and I am super excited to announce this amazing crash course on RAG, that is Retrieval Augmented Generation. In this specific crash course it'll be somewhere around 2.5 to 3 hours, but we are going to discuss everything that is related to RAG completely from scratch. We'll be talking about the entire pipeline from data injection to retrieval pipeline to output generation — how to use LLM models, how to use embedding models, along with what should be the right strategy of using chunking, and many more things. We will be deep diving into both the theoretical understanding along with the practical implementation, and we will initially go ahead step by step. We'll start with the basic implementation, and then as we go ahead in the advanced section we'll also implement the modular coding. The main aim of the modular coding is to link the entire pipeline in a way so that you should be able to understand how RAG actually works, and also implement it in your company use cases.

Let me tell you one very important thing: 90% of the use cases that are currently being worked on in all the companies are specifically related to RAG. So this crash course will be an amazing one for all of you. We'll keep a simple target of a thousand likes and a comments target of 500. So please try to complete it, and yes, go ahead and enjoy this particular crash course. Thank you.

[1:29] So this is a simple definition that I have put up over here, and in this definition first of all we'll try to understand RAG. So first of all let's go through the definition, and then I will give you a brief idea of what exactly RAG is all about. So here you can clearly see that RAG is the process of optimizing the output of a large language model, so that it references an authoritative knowledge base outside of its training data source before generating a response. LLMs are trained on a vast volume of data, as we all know, and use billions of parameters to generate original output for tasks like question answering, translating, and completing sentences. RAG extends the already powerful capabilities of LLMs to a specific domain or an organizational internal knowledge base, all without the need to retrain the model. It is a cost-effective approach to improve LLM output — it's relevant, accurate, and useful in various contexts. So this is just a basic definition; you can refer to this particular definition.

[2:40] So guys, now let's go ahead and understand about RAG. Let's consider that I have a generative AI application, and as you all know, in a generative AI application usually let's say I have an LLM. So this is my LLM. Now usually whenever we have an LLM, what happens is that let's consider that I have a user, a user is asking a query. So this is a query from the user, and before it is sent to the LLM we do add a prompt — this prompt is just like an instruction to the LLM, telling it how it should work — and then based on this we actually get an output. Now, this is a simple generative AI application wherein the LLM is used to generate the content. So obviously by using this

specific technique we give a query, and this LLM — you know that it has been trained with billions of data, different kinds of data available on the internet — based on this it will be able to generate the output.

One of the disadvantages of this, let me talk about the disadvantage of this particular approach. As you know, every LLM that is trained will be trained for a specific set of data. So let's say right now it is 31st August. Let's say this is my LLM model, and this is basically GPT-5, which is the recent model from OpenAI. Now, as you know, when this model was launched, this model may have been trained with data till 1st August. So this LLM will not have any idea what has basically happened in the current world between 1st to 31st August, right? And let's say if I go ahead and ask a specific question to the LLM which is between these specific dates, for any kind of events, the LLM will start hallucinating. So one of the major disadvantages of only using the LLM is that it will hallucinate.

When we say hallucinating, what does this basically mean? It means that even though it does not have the knowledge of what has happened between 1st August to 31st August, any events — even though we ask any question, the LLM will try to generate its own answer because it does not want to look like a fool. That is the best example — it does not want to look like a fool, so it will try to generate some answers, and it will make sure that it shows you an answer that you may also have to believe. That is how it will be written, in terms of the output that we get. So usually this condition is basically called hallucinating. So this is one of the major disadvantages.

The second disadvantage that you have — let's say that I'm using this LLM, and you know this LLM has been trained with a huge amount of data. Now what happens is that I'm running a startup, and let's say now in my startup I'm solving a specific use case, and I have some data which again I need to use along with my LLM. Okay, so let's say that I have some other data, like policies of my company — I have HR policies of my company, I have finance policies — and you know this policy data will not be available publicly, because it is my startup, so this data has been protected. Now I also want to use this specific data and probably create a chatbot. Okay, now how do I do this? Now, one way is that many people will say, "Hey Krish, we can take this particular data and we can fine-tune the model." Right, we can simply fine-tune the model — yes, this is a very good solution, but understand fine-tuning a model is a very expensive process, a very tedious process, because this LLM, whichever LLM we are using, has billions of parameters, and tweaking these billions of parameters usually takes a lot of time, right? So obviously this is a solution, but this is a very expensive solution.

Now, do we have any other way? Any other way — and remember, all this policy data will also keep on getting updated as we run the startup, right? So every time we cannot just go ahead and fine-tune it, like every day we're not going to fine-tune it, right? So we should try to find out a solution — like, how do we prevent this? So this can again be prevented with the help of RAG. Now, how it will be prevented with the help of RAG, I will talk about it. So here, instead of fine-tuning, I'm saying that I will go ahead and implement RAG. Now, you'll understand this only when we understand the pipeline of RAG, which I will discuss in this specific video. So these are the major two disadvantages that you see over here, and yes, there are some more disadvantages which we'll just deep dive into more as we go ahead.

[8:12] Now, what happens if we use RAG, and how are we preventing it? See, RAG is nothing but — it is saying that it is a process of optimizing the output of a large language model, so it references an authoritative knowledge base outside of its training data. Now how do

we solve this hallucinating problem that we have? Okay, so let me just go ahead and draw the diagram again. So here is my LLM, and here is my query. So let's say I am coming up with a user query — so let's consider it over here, and here I'm drawing a user, I'm user. Okay, and this user will first of all give a query. Now what happens is there will be two important pipelines that will be created. As I said over here, we are trying to optimize the output of a large language model, so it references an authoritative knowledge base outside of its training data source.

So, as you all know, this is my LLM, right? This LLM is already trained with a huge amount of data. Now, along with this, I will be having an external database, and this database we basically call it a vector database — an external vector database. Now, you know that this LLM is already trained with some amount of data, and any additional data — let's say my startup data, my policies, HR, finance, whatever data is there — we will try to create a data injection pipeline over here. Now, what will this data injection pipeline be? So let's say I have my data, from this data we will do some kind of parsing, and from this parsing we will do embeddings, and then we will finally store it into the vector store. Okay. Now, whenever we talk about this specific data, this data can be in any format — it can be in PDF format, it can be in HTML format, it can be in Excel format, it can be even in SQL database format, or unstructured format, any format.

So what we do initially, we take this data and we do data parsing. Now here, data parsing is a very important step — I think if you crack this step, then developing a RAG application becomes very easy. Data parsing is all about how do you read the unstructured data or the structured data that is present inside this, and how do you chunk this data — how do you divide the specific data into chunks? Chunking is very important because you need to save this data inside some kind of vector store — this is nothing but a vector store or vector DB. Now, vector store and vector DB is nothing but it will actually help you to save vectors inside this. So once you do the chunking, after doing the chunking, you pass it to the embedding models. Now, in the embedding models, you basically convert text to vectors — vectors is just like a numerical representation for text, so that you will be able to apply algorithms like similarity search, cosine similarity techniques that are already available, right, wherein similar kinds of results based on a specific query can be retrieved from this particular database.

Okay, so here whenever I talk about vector DB — this is my vector DB or vector store. Here we are storing embeddings, and this embeddings will get applied to every chunk. Embeddings is nothing but we basically use it to convert text into vectors. Here we can use different embeddings, like Google Gemini models, we can use OpenAI embedding models, we can use Hugging Face embedding models, and each and every embedding model exists with different costs — and there are also open-source embedding models which will actually help you to convert the text into vectors. Now this is one specific pipeline, which we call the data injection pipeline. At the end of the data injection pipeline, you are able to store the text into vectors inside your vector DB.

Now, how is RAG different from the previous one? Right, so initially you had this data injection pipeline where you are converting all your data into vectors, right, and this data is specifically for this particular startup, and now I have created a knowledge base — so this is my knowledge base, external knowledge base or internal knowledge base, whatever knowledge base I have. And this knowledge base does not exist with this LLM, right? Yes, some amount of information may be available, but not the entire part. Now see the definition — it is a

process of optimizing the output of a large language model so that it references an authoritative knowledge base outside of its training data.

Now what will happen when a user gives a query? Now this query, instead of directly going to the LLM, will go to this vector database, right, and before going here we also need to go ahead and apply embedding, because this query will be converted into vectors — why do we need to convert it into vectors? So that when we are hitting this query to the vector DB, this similarity search is basically applied, and based on this we get some kind of context — we get some information from the vector DB. And now whatever query I'm asking — okay, if I ask, "Hey, what is the leave policy of my company?" — right, now what will happen? First of all, it will go to the vector store, it will gather all the related information that is available over here, and that information, when it is sending it to the LLM, is called context. Now we use this context along with — we go ahead and write a specific prompt. Now this prompt is an instruction to the LLM, and it says that you can use this context to answer the question, and finally you get an output.

This is the entire pipeline. This pipeline is basically called the retrieval pipeline. And this is a very good example of a traditional RAG. Now you may be thinking, "Krish, what about other types of RAG?" Don't worry, I will explain it completely from basic to advanced, with implementation, each and everything, because later on we'll be discussing about agentic RAGs. We'll be discussing how agentic RAGs actually work, each and everything. But I hope you got an idea with respect to this. Now, here you will even not be seeing this particular problem — like, you'll not completely remove hallucination, but some amount of hallucination — if any queries that are asked related to the data that is present in the vector DB, I will definitely get some kind of context and my LLM will give me the output. But let's say that if that data is not present over here, then the LLM can hallucinate.

But here, one best example that you can look at is Perplexity. Perplexity is nothing but it is based on RAG — it is completely developed based on RAG applications. In Perplexity you have connected various retrievers, you are connected to tools, you are connected to web search, and then it is summarizing the output given by the LLM. It also uses various LLMs itself. I'm also planning to start a startup soon, within a couple of weeks I guess, and the kind of application that I'm developing is a RAG application only, and it solves a very good problem for a developer. That is the reason I'm not even able to upload a lot of videos, because I'm pretty much involved in those startups, working on developing a product that India can definitely remember.

Okay, and this is how, you know — this is how things are, and you can basically see how good the pipeline actually works, and this is basically a traditional RAG. Now you may be thinking what all things we'll be discussing — okay, fine, we have discussed about a traditional RAG. In the future classes, what coding will we be doing? Okay, so let's go ahead and talk about it. As I said, two important pipelines we'll go ahead and create — one is a data injection pipeline, and one is a retrieval pipeline. Now, in the data injection pipeline, you'll be seeing that we will be performing data injection. Along with the data injection we will go ahead and do data parsing, then we'll perform embeddings, then we will store everything into the vector store. Then we will create a retriever for this, and whenever a user asks any query, it will be able to give the context to the LLM, and then finally we will be generating the output. So here this is retrieval — this is augmentation, right, this is augmentation over here.

Augmentation basically means — you're giving a context to the LLM along with the prompt to generate the output, right? So this is basically called augmentation, and finally you're generating the output, right, which is nothing but generation.

[18:16] Now, in the next session, how we are going to implement this — first of all I will show you how to perform these all steps in a very efficient way. I will show you how we can perform all these steps: data parsing and embedding. Here we are going to consider different files, like PDF, HTML — you can consider Excel, you can consider SQL database, you can consider any kind of files. Then we'll do document parsing, and we will try to convert this into a "document" — document is an amazing data structure which you can basically use, and you can even parse this, do the chunking, and store it in the vector store, then we'll perform embeddings. Here we will use both open-source and paid embeddings for the same, and then finally we go to the vector store, then based on a user query, how do we go ahead and apply the same embeddings — we are going to see that, okay, and then finally we'll be developing this.

I'm mostly focusing more on making bigger videos so that you don't just have to follow a playlist. I want to basically cover a lot of stuff in one video, so that you should also be able to efficiently cover it instead of covering 50 different videos. Now, when we are doing data injection and data parsing, there are various techniques — we are going to see about optimization, various chunking strategies, context engineering — all these kinds of topics will be coming up when we talk about data parsing, like what is a semantic chunker, how do we go ahead and do the chunking in these strategies, and everything, we'll try to discuss as we go ahead.

[20:03] So we are going to continue the discussion with respect to RAG. Already, till now, we have understood what is RAG, then what are the main drawbacks we are fixing with RAG, and along with that we have also understood how the RAG pipeline is — it usually consists of two important pipelines: one is the data injection pipeline, and one is the retrieval pipeline, which includes these two boxes. Now we are going to go ahead with some kind of practical implementation. Now, the major thing that usually comes to my mind whenever we go ahead and start any new series is how should we cover a specific topic, so that we can understand the coding from basics and we move towards modular coding. So that is how I'm going to implement this entire pipeline — initially we will go ahead with some basic code, we'll try to understand the fundamentals, and then we will start writing more complex code. We'll be using modular coding also. So initially we will write all the code in a Jupyter notebook, then we'll increase the complexity, we'll write the code in terms of classes, reusability, and then we'll try to see how we can actually create the pipeline. So that is how the agenda will probably go ahead.

So two important things that we'll think about: the first important thing is to understand about the document structure. Now, whenever we work with any external knowledge database, any data that needs to be fed into the vector DB, you definitely need to know about this document structure. Why? Because inside this data injection pipeline the first step is data injection. Now whenever we talk about data injection, here we can have any kind of files — we can have PDF files, HTML file, DB file, Excel file. Our main aim is to read all this particular file content, and probably convert it into a structure wherein we can additionally do — we can apply strategies like chunking, embedding, and store it into the vector DB. That is what this entire pipeline is all about. So for that you really need to understand this document structure.

So if you see this diagram, since these two are the main topics that we are going to cover in this particular video — initially we will go ahead with document structure understanding, and then we'll try to build our complete RAG pipeline. In our complete RAG pipeline we have two important steps: one is the data injection pipeline, and the other one is the query retrieval pipeline. Now whenever we talk about the data injection pipeline, let's talk about this in complete depth. So initially you have this data injection pipeline. In the data injection pipeline the first step is data injection — that basically means, let's say you may have different kinds of files, like PDF, HTML, Excel, DB file, or unstructured file — any kind of file format. So in data injection, our main strategy is: how to proceed with reading this particular file, how to perform data parsing, and then finally how to convert this into a document structure.

Document structure. So that is the reason, in this video, as I said, we're going to first of all understand about document structure — how to build this document structure, what is metadata? Now, inside this document structure, you will be learning about important components like metadata, you'll be learning about content, you'll be learning about how the structure of the metadata exists, each and everything, right — so we will be covering completely in-depth, like how these things actually work. Once you understand this, and this data parsing, it is a really important step, because based on this parsing the query retrieval pipeline later on can become much more efficient — you'll be able to get results much more accurately. That is the reason you need to really focus on the data parsing. Now, after doing the data parsing, the next step usually is something called chunking — here in the chunking we convert this entire data into chunks, multiple chunks.

[27:34] So that is what my strategy is, and please make sure to complete the assignment also, and we will go step by step, completely from scratch, so that everybody will be able to follow. So first of all I will go ahead and open my empty folder, and in this, remember, I will be using LangChain, and this is just a traditional RAG for now — in the later stages we will move towards agentic RAG. So from this particular command I will just go ahead and open my command prompt. I will open my VS Code, and from the VS Code the next step will be that I will quickly open my terminal, and let me just go ahead and write `uv` — I'll just go ahead and initialize this particular workspace as my repository. So `yt-rag` is my workspace. Now I will just go ahead and also create my environment — if you're using the `uv` package, you can just write `uv env`. My Python 3.13.2 will be the recent Python version that I'm specifically using for this particular project. And then I will go ahead and create, activate this particular environment.

Now I will go ahead and create my `requirements.txt`. From this `requirements.txt`, let me quickly go ahead and install some of the packages, like `langchain`, `langchain-core`, `langchain-community` — all the things are there. Let me quickly go ahead and install these packages, so `uv add -r requirements.txt`. Along with this I will also go ahead and install some of the libraries like `pypdf`, `pymupdf`. Okay, so these are all the libraries I'll be using — I'll talk about why I'm using `pypdf` and `pymupdf`. This is specifically to read my PDF documents. One example that I'm actually going to show you is with respect to PDF, and then you should also try to create the same pipeline with the help of any other data types — data formats, like it can be JSON, it can be anything as such.

So my `requirements.txt` is filled. Now what I will do is I'll quickly go ahead and create my data folder, and here I will also go ahead and create my notebook folder quickly, so that I can start working on it, and then along with this I will also go ahead and add `uv add ipykernel`, so

that I will be able to work along with my Jupyter notebook. So the ipykernel has got executed. Now quickly, I will first of all start with my Jupyter notebook, and the first thing that I told you is related to document data structure — document, and what is a document, and how it can be very very helpful if you're using it in the document data — in the data injection pipeline. Okay, so I'll quickly select my kernel, and these all things you really need to be good at Python programming language — there cannot be anything that you can skip Python programming language. So my suggestion would be, never do that. Okay, so Python is a must, and this time I'm just going to use some more advanced coding, and it will not be possible for me to write line by line. So definitely I'll go a little bit fast in order to explain to you.

[31:08] Now, as I told you, if I go back over here, in the data injection our main aim is to load some data, apply some chunking, then convert into embeddings, and finally store it into the vector DB. That is what my entire data injection pipeline is all about, right? For understanding this we need to understand a document structure, because all this chunking that is done, you know, the final output will be documents. Now, what exactly is a document data structure? So here I will go ahead and write what exactly is a document data structure. For this I will go ahead and import from LangChain — or to probably show you this. I will be showing you some kind of file, so that you'll be able to understand it. Okay, let me put this file over here. Okay, I have some file over here, and then we'll try to understand. Okay, what exactly is a document structure? See, LangChain document structure — so LangChain document is a kind of data structure which will be able to save some data in some format, where we have two important things: one is the page content, and one is the metadata.

The page content will basically have the content that is present inside that particular file. So if you are reading the file, inside my page content all that detail, all that content that is present inside the file will be available over here, and metadata will be some more additional information of the file — like it can be the file name, it can be how many number of pages are there, it can be what is the timestamp of the file, each and everything. So this way, whenever you read any kind of data, and you convert them, right, in a document data structure, this format will be very very important, because at the end of the day we will be doing the embedding on this particular data and pushing it into the vector DB, and when we do that specific task — pushing it to the vector DB — we will be able to apply different algorithms like similarity search, cosine similarity, and we'll be able to retrieve the results.

So here you can see that all the information regarding this is given over here. So usually LangChain document structure — it has two important core components. One is `page_content`, and one is `metadata`. And here, `page_content` will be the actual text content, where all of it will be very very handy in research papers if you want to probably create a RAG application for research papers or a product manual. So you can specifically use this in LangChain — you definitely have different loaders. Loaders like you have something like a PDF loader, you have a CSV loader, you have a web-based loader, you have a directory loader. Now, all these loaders — what it does is, for the PDF loader, it will be used to load the PDF files, and once it loads the PDF file, it will be giving you the output of the documents in the form of a document structure. I will show you practically also why I'm specifically saying and stressing on this — it will definitely give you all the output in the form of a document structure.

Similarly, in the case of a CSV loader — here we are giving the CSV file, but it will try to convert the entire content that is present inside that CSV into a document data structure. Similarly, with respect to web-based loader, Clarity loader — similarly, there are so many different loaders over here, right? You can use any of these particular loaders to load the data, and at the end of the day this loader will finally give you the output in the form of a document structure. Okay, so I hope you got an idea about what exactly is document structure itself.

[34:47] Okay, so now quickly, what I will do — I will go ahead and start explaining how we can start with the document structure. So for the document we need to import from LangChain — LangChain, there's something called text splitter, and, sorry, LangChain-core — it is present inside `langchain_core.documents`, import `Document`. Okay, now this `Document` — you will be able to see that if you just hover over here, you'll be able to see the class for storing a piece of text and associated metadata. Okay, now if you really want to understand a document structure, first of all I will go ahead and create one document manually — I'll go ahead and use this `Document`, and inside this we will be using two parameters: one is the `page_content` — let's say this page content, I'm writing "This is the main text content I'm using to create RAG." Okay, so I've just basically written some content over here — let's consider that this particular content is coming from a `.txt` file.

Okay, but along with this content, if you really want to improve the search query retrieval from the vector DB, you need to also go ahead and write metadata. So the second parameter that you'll be able to see is something called metadata. Now, inside this metadata, you can write different information, because at the end of the day this is text — you can write, "Okay fine, this is my source." The source is basically coming from `example.txt` file. Okay, then let's say the number of pages is equal to one — total number of pages is like one. I can also go ahead and write some more information, like, okay, who is the author for this? The author is nothing but "Krish Naik." So this is the additional detail that you'll be able to see. Okay, fine, let's go ahead and write "date created" — date created, right, date created, and here I can go ahead and write `24-01-01`, like first 2024 or first 2025.

Now, why are these all metadata really really important? Because once we consider this document, right, once we do the chunking, once we do the embedding, and once we store it into the vector DB — when you're doing the similarity search, you can also apply filters, and that is the most important thing about this. When you apply filters — let's say that I am applying a filter, I'm searching what is the main text content for building the RAG — some information is there. Let's say there's some information related to the RAG. If I ask that particular question, and I say "by author Krish Naik," I just add that particular filter, then it knows from which document to probably pick up, because it is going to apply a filter by using the name of the author, right, and that is why this metadata will definitely play a very important role. Now if I just go ahead and execute this doc, you'll be able to see, fine, I'm getting this particular document — here you can see metadata is there, and as you go ahead you'll also be able to see `page_content`, right? So these are the two main important parameters with respect to this, which everybody can probably go ahead and use it.

[38:11] Okay, now I hope you got a very clear idea about it. Now what I'll do, I will just go ahead and create a simple `.txt` file. Now for creating a simple `.txt` file, what I will do — I will just go ahead and import `os`. And I'm saying `os.makedirs("data/text_files")`. So I'm trying to create this particular folder inside this — I'm creating this particular folder name, and if it already exists I'll say don't do

anything. So as soon as I go ahead and execute it, you'll be able to see, okay, it is going inside the notebook file — I'll remove this and let me go ahead and write `../`, let's see now — you can see over here, `text_files` is present. Okay, so `text_files`, I've just done that — now let me go ahead and manually create a text file with the help of Python code. Okay, so I will just go ahead and use a Python code. See guys, these are all our basic Python code — I don't want to write each and every line of code and make it very big. Our main aim should be to understand concepts, quickly show you multiple use cases, and then try to implement this.

Okay, so now you will be able to see I have created this simple text — I've given the file name something like this. So let me go ahead and write this to it: `data/text_files/python_intro.txt`. And this is some content that is present inside that particular key name. Okay, so this is my file name — you can see this is key — this is my file name, and then here I have specifically my Python content. Okay, here I'm saying for file content in `sample_text.items()`. I'm telling to open the file name, I'm saying that write the content. Okay, so this file path is nothing but my file name. Okay, so if a file is not there, it will try to create `python_intro.txt`. So now if I go ahead and execute this — so it is saying, "no directory." Okay, let me just go ahead and create one file. Okay, `python_intro.txt` — I have to give the path, because there are two files that are over here. One is okay, one file is also over here. Okay, so I'll just go ahead and write `..`. Okay, so now here you can see my sample files have got created — `machine_learning.txt` and `python_intro.txt`. Now what I will do — I've created some sample files, I could have also manually created it instead of doing the code, but I really wanted to show you all the things.

[40:36] Now what I will do — I will show you how to read this particular text using text loader. So one of the loaders that is present inside LangChain is something called text loader. So here I will go ahead and write from `langchain.document_loaders` import `TextLoader`. Okay, text loader — so here we have imported text loader, and along with this — see, if you don't want to also use this, if I execute this, this is also there, before — if I talk about it, right, when LangChain keeps on changing its library here and there. So there, we used to use `langchain_community.document_loaders`, this also we used to use, import `TextLoader`. So any of them you can actually use, unless and until you get a deprecated warning. Okay.

Now the question is: how do we go ahead and read the text? So I'll write `loader =` — I will initialize `TextLoader`, give the path. The path is nothing but the parent folder — we go to the parent folder `data/text_files/python_intro.txt`. So here I have actually given my file name, whatever file name we have actually created, and we can also go ahead and use `encoding="utf-8"`. Okay, encoding UTF-8. So once I do this, okay, and now once I go ahead and read this loader — now what it is giving, it is giving me an object of a text loader. Right, now in order to get the content inside this, I will be using `loader.load()`. Load. Okay, and here you'll be able to see that I will be getting the document. Okay, now let's go ahead and print the document — so I will write `print(document)`. So let's say this is my document. I'm going to print it, so here you can see in the document you are getting metadata, you're getting the entire information, and this is your `page_content`. Now this is what it is doing, right? This text loader by default is giving you the data in the document structure — as soon as it is reading, and here the best part is that you can also see some of the metadata information has also got updated, like what is the source, right, you can still go ahead and manually change more information inside the metadata, but by default the best part is that whenever you're using this — all libraries — then also it will be able to give you the content in the document structure, which is really really

good, because in the document structure you have two important things: one is the metadata, and one is the page_content. So this is with respect to text loader — I have just read the text loader, and I'm able to get this in this way.

[43:22] Okay, now, one more way — what I will do, I will show you with the help of directory loader — like if I have all the important files in my directory, can I read it like that also or not? Okay, so for doing this let's use one more library which is called directory loader. Right, so here you can see `langchain_community.document_loaders import DirectoryLoader`. Now inside my directory loader you can see that I'm giving this particular file — again, this file should be the parent folder — does this, and here I've given the pattern to match — see, this function basically, you can give a pattern to match all the files, then you can use `loader_class`. `loader_class` basically means which file you're planning to load — if it is a PDF one, you can directly go ahead and use PDF. Okay, so what I can actually do is that I can also go ahead and insert PDF files over here — I can also provide this in the form of a list, so that it will be able to read both the content. Okay, so once I go ahead and execute this, you can see here also I'm using the encoding, and all these things. And here you can see, once I go ahead and write `directory_loader.load()`, okay, and here you will be able to see documents. Okay, and then now if you just go ahead and print the documents, you should be able to see this — okay, I'm getting an error to log the progress, please install `pip install tqdm`. Okay, so here we have enabled the parameter `show_progress = True` — let me make it as false, so that I don't need to probably go ahead and install this. Now here clearly you can see that there were two .txt files — I got two documents. Yes. Now further you can do chunking and all, right, based on the number of documents over there — I was able to get it. Right, so this is the most amazing part about this.

[45:14] Now what I'll quickly do is, let me go ahead and create a PDF file also. Okay, so here I have some examples of the PDF file. Okay, so let me quickly go ahead and copy this and paste it over here — reveal explorer data. I have text files, I have PDF files. Now inside this PDF file now my main aim is to read both the text and PDF files. Let's see — so here I have "attention" PDF, this PDF, this PDF. Okay, so this is my one document. Okay, let me go ahead and write the same code — copy and paste it over here, and this will basically be for the PDFs. So for PDF, I will be having, from `langchain` — `langchain_core.document_loaders import PyPDF` — I think `pypdf` is not available over here. Let's see where is this specific library — I'm just checking out the documentation. `PyPDF` — oh yeah, it should be there. So it should be here, inside my `langchain_community.document_loaders`, I have two different types of library: `PyPDFLoader` and `PyMuPDFLoader`. `PyMuPDFLoader` is better when compared to `PyPDFLoader`. You can see, `PyPDFLoader` — loads and parses a PDF file using `PyPDF` library, and similarly if you go ahead and see `PyMuPDFLoader`, it loads and parses a PDF file using this — provides methods to load — this, this is all the information you can see, the differences, which one is better, which one is not better, in the later stages.

Okay, now what I'm doing is, I will give the path over here. So from `data/data` and here you can see the path is nothing but PDF. Here I will go ahead and write PDF instead of writing text loader, I will go ahead and write `PyMuPDFLoader` — let's go ahead and use `PyMuPDFLoader`. I can also include encoding in this, and here what I will do — I will quickly write `pdf_documents = directory_loader.load()`. Okay. And then if I just go ahead and see PDF documents, you should be able to see there are so many different PDFs. Okay, I'm getting an error — `get_text()` got an unexpected argument. Okay, let's remove this — I will not be requiring anything, we don't need to apply any encoding by default. Okay, so here you can see I have got all my documents. Yes.

So how many different files were there inside the PDF folder? One is “attention,” PDF, “embedding,” PDF, “object_detection” — these are some of the research papers, and with respect to this all we are able to see this, and now the best part is that when you’re using PyMuPDF, here the metadata information is completely different — creation date, source, file_path, total_pages, right, format — see, total pages is 15 for the first one, then 27, then 21 — see, you can see it so beautifully, it is there. See, I have also created some of the PDFs there also — you’ll be able to see some kind of author’s name also right, it tries to bring up all the entire source information, and this is your page_content, right? So beautifully you are able to see the entire content quickly, right? So that is what this all PDF is all about, and here at the end of the day, even though we use this specific library, we are getting this in the form of a document structure — it is a list of documents. So if I go ahead and say `type(pdf_documents[0])`, you’ll be able to see, okay, it is of Document type, right — now that is the most important thing.

If you now see that we have understood about document structure — we know how to read PDF and .txt — now don’t you think you can actually easily find out how to probably go ahead and read the Excel, DB, any kind of files, and this is the task that you really need to do. How you’ll do it — just go to LangChain document loaders, right, and you will be able to find out everything over there. Just go ahead and try it out. Try it out. Try it out. Try to see if the document structure that you’re getting is good or not. So here there are so many different things you can just go ahead and try out — if you want, from AWS S3, you want, from AWS S3 directory, go ahead and just install this particular library, but before that you have to do the authentication and all. Once you do this, and once you’re able to do it, you can use any kind of document loaders as you like — but at the end of the day, what is the best thing about this? At the end of the day, you are able to convert everything into a document data structure.

[49:53] Right, now if you see with respect to data injection here, you have actually completed. Now the next step is that I will move towards chunking. I’ll show you how the chunking can be specifically done, what are the different ways of chunking that you can actually do, and then finally we’ll see how we can even convert into embeddings — we’ll try to use open-source embeddings for this, and then finally a vector DB. So yes, I hope you have understood about the data injection part; now let’s move towards the chunking part, where we will understand how we can actually perform chunking, and I have also told you the importance of chunking.

[51:32] Okay, so uh here I will go ahead and start the entire RAG pipeline. Okay, so this pipeline is nothing but data injection to vector DB pipeline. Okay, vector DB pipeline — we are going to go ahead and build this quickly. So, first step, as you know that I already have one data folder over here — so this is my data folder, and I definitely have a lot of PDF files inside this PDF folder itself. So first thing first, what I will do — I will go ahead and create a function, you know, saying that where I will try to read all the documents from this, and I will try to read the data inside this particular document, that is PDF file, and then we may use `pypdf` folder, PDF loader, and then finally convert that into a document. Okay, so for this what I will do, I will quickly go ahead and create a function, and this function will be nothing but — this is a markdown. Let me just go ahead and make a code cell. So, before I go ahead, I want to import all the important libraries that are available. Some of the libraries that I will be noting down over here is nothing but `import os`. Then you have something called `langchain`, `document`, `langchain_community.document_loaders` — I’m using `PyPDFLoader` and all — then you also have this `langchain_text_splitter` and `RecursiveCharacterTextSplitter`. Okay, so I otherwise, instead of

writing in a new file, I will let's go ahead and use, okay, this file is fine, so I will just go ahead and execute this. I will — I don't require the `pathlib` library. So once I execute this, these all libraries will get executed. Now we will be able to use this. Now, since my first step is related to data injection, now whenever I really want to specifically do data injection, what I will do is that I will try to read all the PDFs. So we will read all the PDFs inside the directory. Okay, directory.

Now guys, you need to have some knowledge with respect to coding, so otherwise if I keep on writing line by line, it'll definitely take a lot of time. So here we are going to create a function which is called `process_all_pdfs`. Here we need to give the PDF directory — once you give the PDF directory, we will probably go ahead and take the path. So for this also I will be requiring the `pathlib` library over here. So once we get the path based on the workspace location, here we are going to get the PDF directory path, then we'll list all of it, we'll go ahead and apply this regular expression to get all the PDF files. Then here I'm printing what is the length of the PDF file, and we are processing every PDF file. So here you can see that I'm using `PyPDFLoader(str(pdf_file_name))`, then I'm doing `documents = loader.load()`, here I get the document. Okay, here what I'm doing, I'm adding some more information related to metadata — so here you can see `doc.metadata["source_file"]`, I'm giving the PDF file name, I'm also saying that hey, what is the metadata `file_type`, so this is my new key inside my metadata, to put some more additional information, and finally you get a PDF. I'm just mentioning some more metadata information, so along with this I've put up this metadata information, like `file_type`, `source_file` — now you can add any number of metadata information like you want, and once we read this entire document, we are going to go ahead and store it in this particular variable that is called `all_documents`, which is nothing but a list — it is an empty list. Okay, so once we do this, here we'll be able to see it is returning this `all_documents`. So this function does — from inside a folder it reads all the PDF files, it reads the content inside this, it adds this kind of metadata information, and finally it is basically storing in this particular variable.

Okay, now we call this particular function, `process_all_pdfs`, I'm giving the data folder over here. So once I execute this, you'll be able to see that it has found four PDF files: "attention" PDF had 15 pages, my "embedding" PDF had 27 pages, and "object_detection" PDF had 21 pages, and this is "proposal" one page. Okay, so all the information I have it over here. Now if I go ahead and check my `all_documents`, so if I go ahead and check just this particular variable `all_documents`, you should be able to see that this is my list of documents, right, and the best part is that for every PDF you'll be able to see by default some of the metadata information along with this — you can see there is an author, metadata, keywords, mod-date, all these information are basically present in the metadata information. Now here what we have added — we have added `source`, along with the source you can see we have also total pages is also added, `source_file` is also added, and these are my text which is present inside my `page_content`, right. So for every PDF, whatever is the possible size of the document we have, we are able to read it. Now this is a step that we have done — now we have to go to the next step and perform the chunking.

Now how do I go ahead and perform the chunking? Now I have my all-list of documents, so what I will do, I will just go ahead and quickly create a function, and this will be specifically for text splitting — get into chunks. Okay, chunks I have over here, right. So, first of all, I will go ahead and create a function which is called `split_documents`. Split documents. And inside this documents, I will be giving my parameters. The first parameter is nothing but documents — then I have my `chunk_size=1000`, then I have `chunk_overlap=200`. Okay, so I

have given all these things — now you know how to do the chunking. It is very simple: you go ahead and directly use the `RecursiveCharacterTextSplitter` — and for this we definitely require `RecursiveCharacterTextSplitter`, which we have already imported, I think, right. So on the top you'll be able to see that we have imported this, which is present in `langchain.text_splitter`. So inside we are taking this `text_splitter`, which is nothing but `RecursiveCharacterTextSplitter` — now this is recursively splitting all the document based on the chunk size that is 1,000, chunk overlap 200. Chunk overlap basically means some number of text will be able to get overlapped between two different documents, right, when we are doing the splitting. And here you can see we are also using separators, right — this is just like an empty space, this is an empty space, this is one more separator — this is a new line separator. Now, you tell me in the comment section what separator this is; okay, we can use different separators, you can also use comma — we'll be seeing different types of chunking strategies in later stages, but let's start creating this one pipeline, then you'll get a clear idea about how this entire pipeline works.

Okay, then you have this text splitter — once you specifically have this text splitter, you can actually use this to do the splitting. Right, so now what I will do, I will create a variable inside this, and I will write `text_splitter.split_documents`. So we are using the `split_documents`, and we are giving the documents, and these all are the default parameters that we are giving over here. Now once we do the split, you'll also be able to see what is the `page_content` — I'll just try to display 200 characters from the `page_content`, and you can also see the metadata, right. So once we go ahead and execute this, this is going to return the entire split documents. Now let's go ahead and use this split — let's say here I'm just going to go ahead and get all my chunks. I will be using this function `split_documents`, and let's give the documents — here we are going to give the list of documents, right, like — what are the list of documents? So, list of documents is nothing but all PDF documents. So I will give it over here, and let's see the chunks. Okay, so now if I go ahead and just print the chunks, you should be able to see that my data is basically chunked, right? And you can see that we have split 64 documents into 359 chunks. So these are all my chunks that we have done — that basically means we have converted all our text into smaller chunks, based on the chunk size and the overlap. So like this kind of chunks we have — how much? 359, I guess, how much is it — 359. Initially we had only 64 documents, right — for every page there will be a separate document structure. Perfect. So we have done this, and we have done the splitting part.

[1:00:48] Now let's go to the next step. The next step will be quite interesting, because now if you see from this particular pipeline — right, what are we doing? So here we have done the chunking, but these two are the most important steps — one is the embedding, right, we need to perform some kind of embeddings over here — embedding generation and vector store DB, right. Embedding — you can use any kind of models, but I will try to focus on using open-source models, so that everybody will be able to just try it out. For this what I will do, I will just try to use some kind of modular coding, so I will try to create some classes — for embedding I will create a separate class, and inside this we will try to define different functions, because in embedding, you know, you are converting text into vectors, right, so for converting text into vectors I may define different functions, like loading the model, generating embeddings, that kind of thing, and in vector DB also, again, we'll try to create this as a separate class. So let's go ahead and discuss about this, wherein we work on the embedding part — quickly let's go ahead and see the embedding part.

So for the embedding I will just go ahead and write a markdown — so let me quickly write “Embedding and Vector Store DB.” Right, so we are going to specifically go ahead and implement these two important modules. Now first of all, what I’ll do is I definitely require some kind of libraries over here for embeddings. So for embedding, we are going to use sentence-transformers — we are going to use a model that is available in Hugging Face, and for that I will be using the sentence-transformers library. Along with this, I also want to use some kind of vector store — so this is the vector store I may use, that is FAISS — you can use FAISS, or you can also go ahead and use Chroma. So these are some very good open-source vector stores that are available now — these all libraries will be more than sufficient to get started with. So quickly let me go ahead and install it — so I will write `uv add -r requirements.txt`. So once I do the installation you’ll be able to see. Okay, the installation will get completed — so once the installation gets completed it’ll take some amount of time because we are loading the entire transformers. So here you can see that quickly it has got installed. Now I’ll go again back to here — now once I go over here, what is the first step that I’m actually going to do is that I will quickly go ahead and import some of the libraries that I require, like — I’m importing `numpy`, from `sentence_transformers` I’m importing `SentenceTransformer` — my embedding model will be available inside this, then I’m importing `chromadb`, then we also importing the `Settings` from this — we are importing `UUID`, the reason for creating this `UUID` is that because every record that we specifically insert into the vector DB will have some kind of ID over there, we’ll generate that. Then along with this we will also be importing `List`, `Dict`, `Any`, `Optional`, and since we are going to apply cosine similarity while doing the retrieval from the vector DB, I also will be importing this, and this is available in `scikit-learn`. So let’s quickly execute this. Okay, and till then I will go ahead and create more number of cells.

Now, as I said, for embedding I will go ahead and write one different class — so I will say `EmbeddingManager`. So this will be responsible for doing the embedding part. So first thing is that once I am creating this, for every class that we specifically create, we need to write an `__init__` function. Okay, so `__init__` — so this is my constructor, you’ll be seeing that it handles document embedding generation using transformer. Here we are initializing the embedding manager, and the model name that we are giving is `all-MiniLM-L6-v2`. So this is available in Hugging Face — this specific model, `all-MiniLM-L6-v2`, and this is responsible for specifically converting a text into vectors, and you get somewhere around 384 dimensions.

Then we initialize the embedding manager — then model name is nothing but the Hugging Face model name for sentence embeddings, we are going to use this. Okay, so here we are initializing the model name. We are saying `self.model = None`. Okay, because here later on we’ll initialize this value. This function is very important: `load_model`. So that basically means my next function will be `load_model`. And this model’s work is very simple — it is going to load this model that is `all-MiniLM-L6-v2`. Okay, so I will create another function which is nothing but `_load_model`. Why do we write underscore? This is just like a protected function — if you know about classes, we use something called a protected function, and within this protected function only, within this class only, it’ll be accessible. So here what we are doing, we are using the `SentenceTransformer`, and whatever model name we have, we are loading it. Okay, we are loading it. So `self.model = SentenceTransformer(self.model_name)` — then this will be modeled, loaded, and here you’ll also be able to get the dimension. For that we use a function called `get_sentence_embedding_dimension`, and by default it will be somewhere around 384 dimensions. Okay, that basically means every text will be converted into 384 dimensions.

So once we have this `__init__` function, we have the `load_model`. Now one more function that we require is `generate_embeddings`, right? So here you'll be able to see this `generate_embeddings` function. Okay, so `generate_embeddings` is nothing but it takes the text, that is nothing but a list of string, and it returns a numpy array. Okay, so here it generates the embedding for a list of text — very simple. So here what we are doing, we are basically using `self.model.encode` — this is the function that we have to use on text, whatever list of text we give, and we are also giving `show_progress_bar=True`, so that we should be able to see the progress bar, and we return the embeddings. Okay, now `generate_embeddings` is one function, `load_model` is one function — we have also used `get_sentence_embedding_dimension`, just to get the dimension. Okay, now for this you can either get — I can — you can either create this particular function, or you can also remove it — it is not necessary — but what I did is that to show you much more in a better way we will create this function `get_sentence_embedding_dimension`. So here is my `get_embedding_dimension`, `self`. So here what we are doing, we just written `model.get_sentence_embedding_dimension`. See, instead of doing like this also I can just quickly write this particular function over here. Okay, so sometimes it is not required, you can also — so I will just go ahead and remove it if you want. Okay, I will just remove it. Perfect. So I have these two-three important functions. Now we can initialize the embeddings. Okay, uh sorry, we can initialize the embedding manager — so here I will write `embedding_manager = EmbeddingManager()`. So I hope this is the class name should not be underscore, it should be like this. Okay, now once I go ahead and write this, and once I execute it, this will just go ahead and initialize the constructor. Right, so here you can see it is loading the embedding model, "All MiniLM L6 V2 model loaded successfully," and here you can see the dimension is 384, right, so it has been loaded — so when we're calling this particular function, this is basically getting loaded, right, so my embedding manager now has the model information over here. Great, so I have my model ready, so if you see from this particular graph this entire class has been created — and create this specific class that basically means over here we have our model, embedding ready, we just need to use it.

[1:08:43] Now, similarly, we'll go ahead and create it for the vector store also. Okay, vector store is just like a vector DB database where you can store all the vectors that have been converted by the embedding layer inside it, so that you can apply any kind of similarity search into it. Right, so first of all, let me quickly go ahead and define a class for this also. So, here I will go ahead and write `VectorStore`. Okay, `VectorStore`. Uh remember guys, the code that I'm showing you is very simple if you just see — you need to have some coding knowledge if you really want to become better in RAG. Okay, now we'll go to the next step with respect to the vector store. Now in the vector store we are creating a class `VectorStore`. Again here we are using an `__init__` method. We are giving a collection name — what should be the collection name for the vector store itself? And here the collection name we giving it as `pdf_documents`. We are also giving the persistent directory, which will be this particular directory that is inside my data folder — persistent directory means whatever vector store is basically created, we are going to save it in the hard disk. So here, first of all I'm giving the collection name, I'm giving the persist directory, collection is none, `self.docs_collection = None`. Okay, and then we are initializing the store. Now whenever we initialize the store, that basically means this function will be initializing the vector store itself. Right, so for this we need to create another function — again, see the code, just observe the code. Here we are initializing ChromaDB client and collection. So here we have written `os.makedirs(self.persist_directory)`, whatever directory path is there

— if it already exists we are just going to keep it like that, otherwise it is going to create a new directory. Then we create a client — `self.client`, wherein we are using `chromadb.PersistentClient` function, and we are giving the persistent directory over here. So what it is going to do — it is basically going to create a client which will be having a reference to the Chroma vector store.

Okay, then we go ahead and create a collection. So here we write `self.collection = self.client.get_or_create_collection`, collections — we're giving the collection name and we're giving some metadata information, like what is the collection information. And here we basically create a collection — collection basically means it's just like where we are going to store the vectors inside my vector store, so it'll be stored inside this particular collection name. Then we are initializing this with the `collection.count()`. Okay, so as soon as we execute this, that basically means my Chroma client will be ready and my collection will be created. Okay, now the next function is that usually whenever we create a collection we need to add the documents, right. So for documents we will be creating another function — quickly let's go ahead and create this, because whenever I have a document I will go ahead and create this particular connection. Okay, so here you can see I've created another function which is called `add_documents`. Here we give the list of documents, we apply the embeddings — very simple, add documents and the embeddings to the vector store. And here you can see, if length of documents is not equal to length of embeddings — here you can actually see this. Now we are preparing the data for Chroma. We require IDs, metadata, document text, and embedding list. So now, whatever documents I have — whatever documents I'm getting, I will be zipping it, that means I'm creating a tuple with embeddings, and then I am creating a UUID. Why do I require a UUID? Because it's just like an ID for a specific record, right, and that will be my doc ID. Okay, doc ID variable and I'm appending it over there. Then we are preparing the metadata — whatever doc metadata we get. Remember, we are iterating through these documents, so we have all the information — so all metadata we are putting it over here. Doc index, content length — we are just adding some more metadata information to put it inside my vector DB. Then we get the document content from `doc.page_content`. And we also get the embedding, where we are converting this embedding to a list. Okay, see, two information is basically required over here — if you see from this particular function, one is embedding, which is my `np.ndarray`, right, and this embedding is coming from where — from the previous function, right, `generate_embeddings`, where we have done it, so it's all linked. See, the reason of creating this in the form of a class is because I want to link each and every pipeline, right. So here we are writing `embedding_list.append(embedding.tolist())`. So we have the `page_content`, we have this list. So what I'm doing, I'm adding that entirely into the collection. So for this we require IDs, we require embedding list, we require metadata, we require document text. So whatever we have prepared, we're just adding it over here based on the parameters, right? And finally you'll be able to see how many number of documents has been inserted.

Now quickly, let's go ahead and initialize. Let's go ahead and initialize my vector store. So I'll write `vector_store =` — uh vector store, and I'll initialize this. Okay. So quickly I will go ahead and write `VectorStore()`. So now this is basically going to initialize the entire vector store itself, right. So here you can see this is my collection name, and “existing document in collection is zero,” since we did not add any number of records. Okay, now if we want to add any number of records, we have to call this function `add_documents`, right? So let's go ahead and do that, and let's call it. Okay, now, first of all, you know that I have already done the splitting of the chunks, right? So here if you go ahead and see this, this is my split chunks, right, sorry, that was the

variable — let's see which variable has got saved. Okay, it should be chunks, right? So these are my chunks, right. Now, chunks — what I am actually going to do is that I will extract all the text from that particular chunk, and we'll generate an embedding. Okay, so for that what I will do, I will say I will put a list comprehension. So here now, let's convert the text to embeddings. Okay, we're going to go ahead and do this. And here we are basically going to write chunks — first of all I'll iterate — okay, I will say, "hey, for doc in chunks" — okay, and we are just going to take this `doc.page_content`. Okay, so we are going to take all this `page_content` and basically go ahead and create my text — text variable. Okay, so once I go ahead and do this, you should be able to see this is my text, right — all the text that I have, and this text I will pass it to my embedding manager, right, embedding manager which I have actually created. So what I will do, quickly, I will just go ahead and execute this once again. I have all my text. Okay, I have all my text — now from this we will go ahead and generate the embeddings. Now once we generate the embedding — how do we generate the embeddings? Very simple, we use this embedding manager, which object we have actually created earlier — if you see over here, this is my embedding manager, right, so we are using this embedding manager, `.generate_embedding`, and here I have to give the text in the form of a list — list of strings, right. So here quickly I will call this particular function, `.generate_embeddings`. Okay, and here you will be able to see that I'll be giving my text — then let's store it in the vector database. So after we convert that into an embedding, we store everything in the vector database. Right, so here I will use `vector_store` — vector store, the variable that we have created, `.add_documents`, and this is `add_documents`, this is a function that we have used, and inside this, if you remember, we have to give our entire chunks — okay, whatever embeddings we are specifically applying — okay, so once we do this, you can see this embeddings, whatever we have got, and the chunks, the documents, the entire documents — we're going to do this. Okay, so let's quickly execute this, and I think now my embedding will happen. Now you can see that for 359 text this is happening, and it has got converted into so many number of batches. Uh, "vector store is not defined" — why is it not defined, let's see what I have defined over there — okay, it should be `vector_store`, so this should be the spelling of my vector store instead of that. Okay, so now let me quickly go ahead and execute this. Now inside that same vector store, it'll get executed. Okay, perfect. Now you can see that the total document in the collection is 359. So if you see over here, uh, inside my u, notebook file, inside my data file, here there is something called `vector_store`, and we have done the persistent over here, right? So persistent basically means the vector store is now saved in this particular hard disk. We can just load this hard disk and we can probably go ahead and execute anything as such. Okay, now perfect. Now you can see that we have completed this entire pipeline.

[1:18:48] Now we have all the data available over here in the vector store DB, right, in the form of vectors. But now the main thing is, how do we perform the retrieval? Because retrieval — see, in retrieval what happens is that whenever we have a user query, we have to take this query, we have to convert that into embeddings again, okay, and then we basically go ahead and hit the vector store in the form of a retriever, and then only we get the context. So in our example, first of all we'll try to get till here. Okay, we have a user query, we convert that query into embeddings, then we hit this particular vector store and we get the context. So let's go ahead and create this specific pipeline now. Okay, and for this pipeline, we will try to create a RAG retriever. Okay. So we will try to create a RAG retriever, so let's quickly go ahead and do that particular thing. Till now we have created all the amazing pipelines — we have created this embedding

manager, now we also have this vector store. Now what I will do is that I'll create another pipeline which will be a RAG retriever. Okay, just to get the specific context. So let's go ahead and discuss about that.

So guys, now let's go ahead and create the RAG retriever pipeline. So first of all, what we are going to do is that I will go ahead and create a class which is called `RAGRetriever`. Now this `RAGRetriever` class — you will be able to see that it handles query-based retrieval from the vector store. So inside the constructor we will be giving two important parameters. One is the vector store, and one is the embedding manager. And if you remember, we have created both this — we have created the embedding manager, we have created the vector store manager, right? Now, after giving this, we will be initializing two class variables, that is `vector_store` and `embedding_manager`, and we'll be assigning with this. Now, whenever we create a retriever, one thing you really need to understand: this retriever is actually built on top of a vector store, and retriever is nothing but it is a simple interface based on whatever query we get, this retriever is just going to give you the response back. Okay, and this retriever is basically a kind of interface which is connected to the vector store and chart.

Okay, now the next step that we are going to create is another function which will be called `retrieve` function. Now this is really important because this `retrieve` function's main work is to retrieve based on a specific query. So let me go ahead and define the specific function — now this function, again, see, to write it will definitely take a lot of time, so we will try to understand this particular function. Okay, so here a `retrieve` function you can see we are giving query, we are giving `top_k` — how many top-K results we want — and there is also a `score_threshold` value, by default it is 0.0, and this function is basically going to return a list of results. Okay, so here you can see, “Retrieve relevant documents for a query,” arguments are the search query, top-K documents, and score threshold, and it returns a list of dictionaries containing the retrieved documents and metadata. At the end of the day this function is actually going to help us get this specific context. So you'll be able to see over here we are using that same `self.embedding_manager`, and we are calling this `generate_embedding` function — now if you remember, this `generate_embedding` function is already defined in my embedding manager, right. So if I go on the top, so here is my `generate_embedding` function, and this is nothing but basically using `model.encode`, and you're giving the text, and it is converting into embeddings. Yeah, so that is the reason we are basically using this, because at the end of the day, whenever we get a query, right, so let me go down over here inside this `retrieve`, whenever we give this query, first the query needs to be converted into an embedding — right, so this query that is given, we need to apply embedding for this also, so that we can do a similarity search in the retriever itself. Right, so first the query is basically converted into a vector by the help of `embedding_manager.generate_embedding` function.

Then we are going to use the `vector_store.collection`, and we are going to use this `.query`, and here we are going to give our query embedding, which is nothing but this embedding in the form of a list, and then we are also going to give the top results. So by using this, this is basically going to hit the vector DB, whichever vector DB we have initialized, and it is going to give you the results. Once you get the results, the results internally there will be a key which is called `documents` — okay, you can get document information, the metadata information, the distance information, and some of the IDs information. So all the specific information we are using it, and here you can see very similarly what we are doing — we are using all these parameters like ID, documents, metadata, and distance. We are

zipping it — zipping it basically means we are just trying to create a tuple over here, and then for every value we are just trying to calculate the distance, right, “1 minus distance,” 1 minus distance will basically give you the similarity score — like how similar those text data is basically coming up outside this vector store. So we are creating the similarity score, and if the similarity score is greater than the threshold, then what we do — we basically add this inside my text, context, documents, and context_documents is basically created in this particular variable which is nothing but retrieved_docs, which we have kept it empty over here. Okay, so all the information we are just trying to add it over here, so that we’ll be able to see it, and finally we return that retrieved_docs. So, if you say step by step, we’re not doing anything like — not very complex thing — we are getting the user query, we’re converting this into embeddings, we are hitting the vector store, right, then we are getting the response, okay, once we get the specific response, that context we are putting it in the form of a list — if you just go ahead and see the code, that is how things are happening. Okay, so this is one of the very important functions that you’ll be able to see.

Now here what I can do is that I can quickly go ahead and create a variable called rag_retriever, and I can call this same class. So if you see over here I will use this same RAGRetriever over here, and let’s give our vector store — vector store which I’ve defined it earlier, which is my vector store manager, and then my embedding manager. Once I do this I should be able to see this — okay, it should be vector_store, right, so now you’ll be able to see this is my rag_retriever — it is an object of this. Now if I call this particular function with a query, right, I can call .retrieve with a query. So let’s go ahead and do this. Okay, so here I will write rag_retriever.query, sorry, .retrieve is my function. Okay, so here you can see quickly, this is my function retrieve, right, and I need to give a query. Now let’s test for a specific query — I’ll say, “Hey, what is attention,” because I know inside my data there is a PDF file which is called “attention,” or I have also created some kind of “proposal” over here, “embedding” — some files are there. So we’ll try to execute this. So here you can see, as soon as I asked “what is attention is all you need,” now it is giving me the top-K — for all it is printing all the information, and it is generated embedding for one text — right, and the text shape is 1, 384, because I have used the embedding that is called all-MiniLM-L6-v2, that creates a 384 dimension. Now, once we go ahead and apply this particular function, right, this function it is basically getting the results over here and we are printing that same thing, right, and at the end of the day we we we can also go ahead and return this retrieved_docs. Okay, so in short, this function is going to give me all the retrieved docs, so this is the retrieved docs — you can see content, metadata, author, so these are my context information, so here you can see, “attention function can be described as a mapping of a query as a set of this one,” and this entire thing is basically the context. So from this particular diagram here you can see easily, we are able to get the context, right, and this is nothing but this is your context.

Now let’s try some more things. Okay, I will just go ahead and open some PDF. Okay, um this is some very new research paper, “embedding,” technical report. Okay, uh we’ll search for any topic over here — embedding model training. I’ll just go ahead and search for “unified multitask learning framework.” Okay, because this information also we have put it over there. So here I’ll go ahead and create one more, this one, and I will copy this entire code. Okay, quickly, and this is the query that I’m actually going to give, that is nothing but “unified multitask learning framework.” So if I go ahead and execute this, you can see that I’m able to get this, and then you can see content, benchmark ranking overall, both the leaders’ effectiveness of our approach — so we are able to get the response

very very much quickly, right, and this response is basically coming from the vector store, right, in a very similar way, very easy way, we are able to get the specific response over here, right, and let me tell you, right, this is the most easiest way, like how things are basically happening over here right now.

Uh, what we can do is, see, if you know, if you have created all these things right, till here you have created — now the further step is that you have to just integrate LLM with the specific context. Okay, now for this LLM with this specific context, what you can do is that you can directly take this particular context and give it to the LLM, and that is what we are going to see in the next video. But in this particular video we saw the entire thing, the complete RAG pipeline from data injection to the vector DB pipeline. Right, now you can go ahead and write any kind of queries, and definitely with all this information, here you can see similarity score is also coming up, distance is also basically coming up, all the information you're putting it over here, and we have also used modular coding, right. Now in the next step, what I'll do, I will take this vector store, and we will go ahead with the next integration, that is LLM and output — which I will say it as a retrieval pipeline, but this entire data injection pipeline with this query retrieval we have actually created. Now the next two steps will be this one, and after doing this we will try to convert the same code, whatever same code we have basically written over here, in the form of modular coding — we'll try to see how we can put this inside our source folder. So here what I will do, we'll quickly create a source folder, and inside the source folder I will show you how we can take this entire pipeline and how we can actually create it in such a way that we have a kind of pipeline over here — pipeline basically means, from data injection to vector embedding, how in a sequential way we can actually go ahead and call it.

[1:30:23] Hello guys, so we are going to continue the discussion with respect to RAG. Till now we have already discussed about the entire data injection pipeline, and with the help of a user query, you know, we are also able to retrieve the context. We have completely implemented this first pipeline, that is called the data injection pipeline, where we did the data injection, we did the chunking, then we converted the text into vectors, and after that, you know, we were able to probably store everything inside a vector DB, and we also persisted it in the local directory so that we can always read it whenever we definitely want. Okay, based on a specific query. Now we are going to go towards the second pipeline, that is the query retrieval pipeline, wherein we are also going to use LLM with it. Okay, so here we are going to specifically use LLM models, and this LLM models will actually help us to generate a summarized output. Okay, in the RAG, so the entire pipeline will look something like this. And uh when we talk about this query retrieval pipeline, we are specifically talking about something called augmented generation. Okay. See, in retrieval, RAG basically means Retrieval Augmented Generation, and this augmented generation, how does it specifically work? Okay, so let's consider that this vector DB is already ready, and you know that how did I create this particular vector DB — by following this particular pipeline, right? Now once we follow this pipeline, the data is stored inside the vector DB. Now, whenever a user gives a new query — okay, it has a new query related to the documents that are already ingested inside the vector DB — then what we do, we take up this query, we apply the same embedding, and in this particular embedding what we do, we convert the query to vectors, right, and then from this particular embedding we hit the vector DB, we get the context, and then whatever context we get, along with the prompt engineering, like basically with a simple prompt we give that instruction to the LLM. Right, so prompt is just like an instruction to the LLM, like how the LLM should basically work. Now once we are doing this, right, this

step is basically called augmentation — okay, this step is basically called augmentation, wherein we are giving — we are taking the context and along with that we are also combining it with a specific prompt, and finally you'll be able to see that we'll generate the output from the LLM. And this step is nothing but generation — right, this is the retrieval step. So here I have my retrieval step wherein we are giving a query, we're converting that into vectors, and we are hitting the vector DB. So you really need to understand the entire concepts with respect to RAG. Okay, so let's go ahead and implement this entire retrieval, query retrieval pipeline, along with the LLMs.

[1:33:24] Now here we also going to go ahead and set up the LLM. So guys, now let's go ahead and implement this with the help of practical implementation. So here we are going to integrate vector DB context pipeline with LLM output. As suggested, we are going to implement the augmented and generation. Now first, first of all, what we are going to do is that I'm going to use my Groq API key. Okay, so I have updated the Groq API key over here in the .env file, and, you know, here we are going to probably go ahead and create a simple RAG pipeline. Okay, with the Groq LLM. Okay, so first of all what we are going to do is that, again, if you remember, in our requirements.txt we will go ahead and import these two libraries, that is called langchain-groq, and then you have python-dotenv, okay, and then after this we will go ahead and quickly initialize from LangChain-Groq, import ChatGroq. Okay, along with this I'm also going to go ahead and import os, then from env I'm going to use load_dotenv, so that we import — we load the entire environment variables, then the next thing is that we will go ahead and initialize the Groq LLM and set your environment Groq API key inside this. Okay, and in order to do this, again, here you'll be able to see I'm using groq_api_key = os.getenv(...), something like this. Okay, if you just go ahead and call this sometime — my suggestion would be that directly don't call from getenv, initially you can directly test it by pasting the environment keys directly over here. Okay, so here I will go ahead and paste it — otherwise you go ahead and replace it. Just for testing purposes I'm actually doing this. Now we'll go ahead and initialize our LLM model, ChatGroq, and here I will use my Groq API key is equal to API — sorry, Groq API key — okay, and then model name is gemma2, temperature I will select as 0.1, and maximum number of tokens it will generate is 1024. Okay, so this is my LLM — we have initialized the Groq LLM.

Now the second thing is that we will quickly go ahead and create a simple RAG tag function, and this is going to integrate everything from retrieve context plus generate response, and if you remember, guys, here is my retriever class, like the previous session — we have already seen how this RAG retriever was actually created, we created a class for that. Okay, so here we are going to probably take two different parameters inside this — we'll first of all define a function called rag_simple, and then here we are going to go ahead and give our query, then we are going to go ahead and give our retriever, LLM, top-K is equal to three. Okay, and then over here quickly let's go ahead and first of all retrieve the context. Yeah, so we'll going to retrieve the context — so here I'm going to write results = retriever.retrieve(query, top_k=k). Okay, so here you have this query and top-K is equal to K. Okay. And then, uh, we are just going to get the context — or I'll go ahead and define my context. Inside this context I will say that hey, whatever information I'm getting from my results, right, just go ahead and combine everything and put it inside this. Right? So here I'm saying that hey, for doc in results, whatever content I'm getting, I'm going to join it with a double new line over here. If results are this empty, we are just going to keep it as empty. So this is my context over here, right, then uh I can still go ahead and write one more condition saying that hey, if not context, okay, we just going to go ahead and return, saying that “no relevant context found

to answer the question,” and then we are going to generate the answer using Groq LM. Okay, and now I’m just going to go ahead and define prompt — obviously I required a prompt. If you remember, here I can again use a prompt template also, I can directly use a prompt over here. So here, with respect to the prompt, I will give a query saying that, hey, this is what you really need to do, you need to go ahead and answer this specific question and you should probably get a response for that. Right, so here what I will do, I will quickly go ahead and paste it — “Use the following context to answer the question.” So here you can see, “Use the following context to answer the question,” “question,” concisely. Okay, and here what we can basically do is that we can just go ahead and do one thing on over here quickly — I’ll say just put “tab.” Okay, so, “use the following context to answer the question precisely or concisely.” So here I have given the context, here I’ve given the query. Okay. Now the next thing after this is that we will go ahead and create a response. So response is equal to this — this time we going to use `llm.invoke`. Okay, and here uh let’s go ahead and put something like `prompt.format`, and here we are going to write “context is equal to context” and here you have “query is equal to query,” whatever query I have. Okay, and then we go ahead and return the response, `.content`. So once we do this, uh, then we can specifically call this particular function. Okay, so now what we are going to do is that I will just go ahead and write “answer is equal to `rag_simple`,” and let’s say I go ahead and ask a question — “What is attention mechanism?” Okay, and here I need to give my RAG retriever along with the LLM, and then we can go ahead and print the answer. Okay, so here you can see “attention mechanism is a function that maps a query,” in this right, and we are able to get the answer over here — this is really good.

See, a very simple pipeline where I have initialized my LLM model, I’ve defined a function, and then this function, what it is doing, first of all it is hitting the RAG retriever, retrieve function, it is getting the context, it is combining the context along with the prompt, we are hitting the LLM. So if you remember, we are we are just following this entire process and generating a proper output, right — if that particular output is available inside the vector DB. Right, now guys, uh, what we are going to do is that we are going to enhance the RAG pipeline — the simple RAG pipeline that we have created over here, okay, we’ll enhance it in such a way that it will have more amazing features in it. Okay, so now we’re going to go ahead and create an amazing enhanced RAG pipeline, and this is the code — so now you can see over here, we have a function called `rag_advanced`. I’m giving a query, retriever, LLM, top-K elements, like how many we want, minimum scores, `return_context` is equal to false. So here you can see that, um, before we were simply like we were just combining the context, we are putting the information in the prompt, and we were probably generating the response. In this, what we will do is that here we are going to generate this entire pipeline with some more additional features, like what all additional features we’ll be requiring. See, here we are directly getting the answers, right, but we do not have much information about the source, about the context over here, right. So here what we are doing, we will return answers, sources, confidence score, optionally full context. Okay, so first of all, again, the code will be similar where we are retrieving the context, so this becomes my context — when we are retrieving it from `retriever.retrieve`, and then uh I have written, “if not results,” “if results are empty,” we are saying that “no relevant context found,” and here we are giving sources is blank, confidence is 0.0, and context is blank. This context is basically coming from the vector DB. Let’s say that if we are getting some kind of results over here, we are combining all those results and we are preparing the context over here, and then we are adding sources. See, this sources, which is the

list, here we are adding metadata information, source file, right, and along with that you can see metadata page number, from which page number you are able to get, then what is the similarity score, and here what I will do is that I'll just try to go ahead and, you know, display at least 300 length of the content, right, so up to 300 characters we'll try to display, and then we are going through each and every document that is available inside this results, then we are going to calculate the confidence — uh, we are actually getting that information in this `doc["similarity_score"]`. Here is my prompt — in this prompt we are giving context, query, each and everything, and we are invoking it, and the output will be in this format. So let's now go ahead and execute this RAG advanced function. Here I've given all the information, like I've asked "what is the attention mechanism," "what is RAG retrieval," like "RAG retrieve," I'm giving over here, LLM, `return_context` is equal to true, minimum score, all these things is given, right. So now I'll go ahead and execute this. Now as soon as I ask "what is attention mechanism," here you'll be able to see that I'm getting this particular information, right, and it is also giving me the source information — which number, page number, what is the score and what is the preview information, along with that here is my final information that you can see right where we are displaying the first 300 characters. Let's say that I go ahead and change my question — okay, I I ask something else, I'll say, hey, "attention mechanism was one of the thing," but if I go ahead see my data, my PDFs — okay, I will go ahead and ask something else. Okay, let's see what I can ask. So I'll go to "embeddings" PDF, I'll say, okay, and then let me search something else, right, I will say "hard negative." I'll ask this question, "hard negative mining techniques." Okay, so I will go to my question over here, "hard negative mining techniques." Okay, and I'll go ahead and search this thing from my vector retriever. So here you can see that I'm able to get this entire information — the test is several hard-candidate embeddings, "NV retriever" — all these information, and again you can see that "embedding.pdf" PDF page 4, I'm able to see all the information along with the context, right, so this is uh really amazing, and here we have just created an "answer-source" RAG pipeline — why we say this as answer-source RAG pipeline, because here we are providing information related to answers, we are providing information related to confidence score and each and everything.

Now let me just show you one more amazing way — and this is also an advanced RAG pipeline, but this time I will tell you to probably go through this particular code and tell me — so here what What we doing? We're doing streaming, citation, history, and summarization. So all these things we have included over here, and you can just go and search for this, and you can see the answer. Okay, "final answer: no relevant context found," because that question may not be there. Okay, I will just go or let me just change this minimum score to 0.1. I think we should be able to get something. Still nothing. Uh, let me change the question — let's say "hard negative mining techniques." And here we are just going to go ahead and display this particular output. Okay, so now you just go ahead and explore this. Okay, I'll keep this for you at least to see some kind of coding. Okay, so here uh we are not able to get anything as such. Uh let's see, "advanced RAG query hard query to top query," "summarize" is equal to "true." Uh no relevant, this one. Let's see that I go ahead and ask "what is attention is all you need." Okay, I'll go ahead and execute it. So here you can see that I'm able to see all these particular answers over here, right. Yeah, for some of the queries this will not — it is not giving — there may be some problem with respect to the context size, but it's okay, you can try out with different different things. If it if something is not coming then we'll try to optimize that also as we go ahead, we'll try to see this. So here we have seen three amazing RAG pipelines. One was

a simple RAG pipeline, here was an enhanced RAG pipeline, and here uh in the last one we have made sure to put streaming, citation, and history and summarization with all this kind of information over here. You just go ahead and check it out, all the information, and just see the code, I think you should be able to understand it. So overall, uh, if you see I hope you were able to understand this particular video, and uh yeah this was about RAG pipeline. Now, in the upcoming videos, what we will do is that we will try to create some modular coding, because see, here the entire everything is basically created in one Jupyter notebook file. So guys, now it's time that we implement the entire RAG pipeline in the form of a modular structure. Already in our notebook we have seen about PDF loader, embedding, and RAG — wherein we discussed how to probably go ahead and create the entire data injection, and how to probably store all the information into the vector DB, and finally you're also able to make the query, right, along with that I have also shown you how to work with Typesense, which was an open-source vector store itself, which was also again amazing for searching anything in a quicker way. Right, now all the kind of implementation that we have done, what we are going to do is that I'll try to show you how in a modular way you can go ahead and integrate this in a form of a pipeline. Okay, so already we have this source folder — now inside this source folder, what I am actually going to do is that I'll go ahead and create `__init__.py` file. And after creating this particular file, what is the next step is that I will go ahead and create all my components — important components that will be required in order to create your RAG pipeline. The first important component is nothing but data loader — right, `data_loader.py` file, right, so this will be my first component, because initially we need to load the document, we need to do the chunking, and then we need to probably go ahead and store it into the vector store, right. So inside my data loader, you know, I will just try to go ahead and read all the documents that is actually required. Okay, then after this, the next step should be your vector store — right, now the vector store, what vector store we are basically going to use — uh, so for that I will be creating my another file, so here inside my source I will go ahead and create one more file which is called `vector_store.py`. Okay, so this is my next file that is basically created. Okay, uh, along with this, uh, while actually inserting anything into the vector store, I also need to probably go ahead and do some kind of embeddings, right, and uh I will try to show you some open-source embeddings that we are going to use. So for that I'll be creating my `embedding.py` file, and finally, uh, the last file that I really want to create is something called `search.py`. Now my entire RAG pipeline needs to be integrated in such a way that there should be a linkage between all the specific files. Now the first case is that I will go ahead and start working on `data_loader`. Now, you know, data loader work is nothing but it should be reading this particular data. Okay, it can be from any source itself. Um, we will try to read this specific data itself, right? So for this, what I'm actually going to do is that I'll go ahead and import some of the libraries. So quickly I will go ahead and import these all libraries, like `pypdf loader`, `TextLoader`, and all. Okay, so I'll start working on this because I need to form a pipeline itself, right? So inside this particular file, my main code should be in such a way that I will go ahead and read all the documents, let it be of a PDF, text loader, or CSV. Okay, here I'm also going to give you some of the assignments, because in this entire series of videos we have discussed about this. Okay, so quickly what I'm actually going to do is that I will go ahead and create one function which is basically called `load_all_documents`. Now, see this. Okay, so here I'm just going to go ahead and write this function. Now please have a look onto this particular function — this function's definition is `load_all_documents`. I'm given the data directory. This should be in the form of string format, and it is returning a list of anything, right — of any kind of data type. Now, the main important thing about this

function is that it loads all supported files from the data dictionary, and converts to LangChain document data structure, because as soon as we read any kind of data like PDF, CSV, TXT, right, we need to go ahead and convert that into a LangChain document structure, then only we'll be able to apply the chunking. Okay, so here you can actually see that I have used data path of the — data directory itself, the data directory I will be giving in the runtime, and obviously by just seeing this, the data directory is nothing but data itself. Okay, now this is the code specifically to read all the PDF files. Okay, so here I have created a list documents which will be storing all the documents itself. Uh here we have used data path glob function, and here I have used this pattern, this kind of regular expression to match all the PDF files. So what it will do is that inside this data directory it will start looking for all the PDF files. So inside this, you know, that in the inside my PDF folder there are some PDF files. So it is going to go ahead and read all these particular PDF files. Okay, so once it reads the PDF files, uh we will be having those PDF files over here in the form of a list. Okay, then what we are doing, we are writing "for PDF and PDF files," we are going through every PDF, and then we are using PyPDFLoader to read the content inside this, and we are using loader.load(), and finally I get all the information over here, and we are going to extend that documents. Now this is just an example of PDF files right now. Same thing you can also do over here for text files. Okay, text files. You can also do it for CSV files, right? See, similar kind of code is basically suggested by GitHub Copilot, but I really want to give you an assignment. Okay, so this will be for CSV file — this can be for SQL files, any kind of files that you really want to work with, you can go ahead and write that particular code and keep on appending inside this particular documents. Okay, so as soon as you do that, automatically you'll be able to do this specific stuff, and you'll be able to get all the documents. Okay, now what I will do just to test it out whether my PDF files is working fine or not, I will just go ahead and create one app.py file over here. Okay, now inside this app.py file let me go ahead and import some of the libraries. So first of all I need to read everything over here, right. So I have written "from source dot data loader import load_all documents." So this "load all documents" is nothing but this is the same function that is present inside my data loader. Okay. And then from "source dove vector store, files vector store, and RAG search" — I will create in the later stages, so right now I'll remove this. Okay, now let's try to test the example. So "example usage," I will write "if name main," okay, and then here I will go ahead and write "documents is equal to load all documents," and I'll give my data folder. Okay, data folder. Then what I can actually do is that I can just go ahead and print my docs. Okay, if you see inside this data loader, what this is returning, right, now it is not returning anything, so what you can actually do is that from here, so here what we are going to do is that we are going to return the specific documents over here, so that we should be able to print that particular documents over here right now — what I am quickly going to do is that I will just go ahead and write "open command prompt" — okay, and here I'm going to go ahead and write "python app.py" — now let's see whether it'll be able to read the PDF files or not. Now here you can see it has found four PDF files, all the PDF file URLs are over here, and you are able to see that it is also able to see all the content that is available inside that particular documents, which is good, right, and this is basically in the form of a document data structure I guess — yeah, so all the information is basically happening, so that basically means, so clearly I can see something really amazing over here is that my entire data — the PDF code that we have written — is working absolutely fine. Okay, now uh comes the next step. Now the next step you should probably start thinking whether we should basically go ahead and work with embedding, so that to do the chunking and all right. So here uh I will go ahead and

start working on embedding. Now inside my embedding what we are going to do is that I'll be importing these libraries — now these all are same thing repeated, but here I'm using classes and function definition — so here you can see that after reading all the documents, after loading all the documents, I'm going to use sentence transformer, recursive character text splitter, and here you can see I've defined a function uh class called "embedding pipeline," right, the model that I'm going to use is "all-MiniLM," uh LM L6 V2, chunk size is nothing but 1,000, and chunk overlap is nothing but 200, then here we are writing self dot chunk size, chunk self overlap, and then we are also initializing the sentence transformer. Now in the next function that we are going to go ahead and do is nothing but uh we are going to go ahead and create a function which is called "chunk documents." Now inside this "chunk documents," we are giving the documents which can be a list of any documents. Here we are applying recursive character text splitter based on all these values that we have initialized. Along with this we have also used different different separators, if you're interested, or you can directly use this blank separator. Okay, then you can see that I am also using the "splitter dot split split documents" over here, and then you will be able to see the remaining chunks over here itself. Okay, now this is for any document that I pass inside this particular function, right, but one thing is very important is that because after the chunking is done, right, you need to also convert that chunking into vectors with the help of this particular model. So for that I will be creating one more function which is called "embedding chunks," right. So here what I will be doing is that I'll create this particular function called "embed chunks." Here we will take this chunks. So what happens is that first the "load all documents" will be called, right, after that the "chunk documents" will be called, wherein all these documents will be chunked, then all the chunks will be passed through our model to probably convert that into a vector embeddings. Right, so here you'll be able to see "self dot model dot encode." So show progress bar is equal to true. Right, so here what we are doing, we are reading all the page content, and we are performing the embeddings, and finally we return the embeddings over here, right, so this is what we are actually doing right. So two important functions, one is "chunk documents" and one is "embed chunks," inside a class called "embedding pipeline." Now the same thing you can go ahead and test it in your "app.py," right, so in the "app.py," what you are going to do is that here um I will just go ahead and, go ahead and just a Okay, let me go ahead and initialize just a second, uh the embedding pipeline. Okay, so here what I will do, I will go ahead and write "from src dot embedding import embedding pipeline." Right? And once you do this, I will go ahead and initialize the embedding pipeline. Okay? And then I will just go ahead and give this right. So this basically becomes my vectors — sorry, "embed chunks," it is there right, so "embed chunks" before that I need to chunk the documents. I also did not call the "chunk documents," so let's first of all call the "chunk documents" over here, okay, and then this will basically be my chunks, and finally you can also go ahead and write over here as my "chunk vectors" — "chunk vectors" is equal to, and here uh you can go ahead and use the same embedding pipeline dot embed chunks, right, and finally you can go ahead and print the chunk vectors. So once you do this, that basically means you'll be able to understand whether the chunking is happening or not. So let's quickly run this particular file again, and now you should be able to see the chunking that may be happening over here. Okay, so it'll take some amount of time because it is going to load all the documents again. Okay. And then the chunk document function is going to get applied over here. The chunk documents what it does is that it just going to apply recursive character text splitter on every documents that we specifically give. Right? And once we do that you'll be able to see that it is loading. You can see all the things are happening over

here. 21 PDFs, one PDF like 21 pages, PDFs is over here with respect to this proposal load embedding all models splitted 64 documents I got into uh 359 chunks you know, and then we basically go ahead and store this. Now the next step is that after this uh I will try to create a vector store and uh we will try to save those embeddings also. Okay, so here you can see all the chunks is uh vectors are visible over here right. So this is really really good. So just just imagine right in a pipeline it is specifically working one by one right it is it is working over here and that's that's the best part out here right now the next step is that what I will do is that I will try to create some more functions uh which can be for save and load uh like if I want to save this entire chunks how do I go ahead and save it you know u what do I save it each and every information that you'll be able to see over here Okay. Now, uh this was about uh the two important pipeline which is basically load all documents and uh embedding pipelines with uh two important function. One is chunk documents and one is embed chunk. So guys, now the next step is that what we are going to do is that now already we have created this embedding pipeline, right? Now let me do one thing because after performing the embedding, we also need to store it in some kind of vector store and it should be persistent in any kind of directory or in cloud. Right? So for this I will start working on this vector store. py file and here I'm going to use some code. Now you can see what all things I'm actually using. So I'm using the sentence transformer and embedding pipeline over here. First vector store is the class name that we going to use. Uh I'm going to specifically use FS. Uh here we are going to use the same model. All mini l6 v2 chunk size everything is over here. And uh we are also making some kind of directories. The persistent directories like fire store should be the name and then here you'll be able to see I'm initializing the embedding model sentence transformer and all now the first step is that build from the documents now see here uh the same code we will go ahead and write what we had written in embedding pipeline right so here we are initializing embedding pipeline model dot self embedding model chunk size and I've given the chunk documents embed document embed chunks I've got the metadata and I'm adding all these embeddings inside my vector store and once I use self save Save. What is this self dots save? Save is a function which is going to save all the vector inside this index dot pickle files. Right? So metadata is basically getting saved in pickle file and files. index will basically be my vector store which will be in the persistent directory. So that is the reason I have written files right index self dot index files path right with open metame this and all information is there right. So this same method is basically there add embedding method is over here. Add embedding is nothing but it is basically taking it it is adding as a index flat tail two. So these are some basic stuffs when you actually work on this. Along with that I've also created two more function load and search. Load and search what it does is that it will actually allow you to load the files index the vector store. Okay. And will uh load it in the read byte mode and then with the help of search and query you should be able to ask any kind of queries that you have. Right. You can also use this query method. Uh here you can see we have written self dot model dot encode with respect to the query test as type float 32 and with the help of query search you'll be able to get the output. Okay. So this was about my vector store. Now in the app. py what I am actually going to do I will just go ahead and make some changes. Okay. Now what what are the changes that I will be making? Okay. Instead of calling this two, okay, I will just go ahead and write store is equal to first of all let me go ahead and initialize this files vector store. So source dot embeddings files vector store here okay and here I will go ahead and initialize this and let me go ahead and give the path name. The path name is fires h o r e. Okay. Now initially if this p path is there then it is fine. Otherwise it'll go ahead and I'll just go ahead and write store. build

from documents of all the docs. That's it. Now if I do this it is just going to go ahead and for the first time it is going to build it. Okay it is going to build it. So let's see whether it'll be able to build it or not. So here I'm going to clear the screen. Python app.py let's quickly see this now it is going to read first of all it is going to read it then this is fine loading perfect load all the PDF files. Perfect now the chunking will happen automatically and it'll save it in the vector store inside that particular folder that is files let's see now it is generating 359 chunks all the steps are almost same what we have discussed from starting but this is A very super cool way of building something. Right? Now you can see save files index metadata to fire store vector store also. So here you can see fire store is there fires. index and metadata. pickle right now we need not run it each and every time right uh because uh once we have this right from the next time what we can do instead of always building unless and until you have a new documents I can also go ahead and write store. load okay if I go ahead and write store. load. Okay, I should be able to print anything that I want, right? Let's say I will go ahead and print something like this. I can use the same query method that we had. What is attention mechanism? Top K is equal to three. Right? So once I do this, you should be and this time I don't think so we need to also read any kind of documents also over here. Right? So I'll comment it down over here. This also you can uncomment it if you really want to or you can also give another conditions. Now what it'll do, it'll directly go ahead and read from the vector store. It'll pick it from the persistent directory and it'll give you the output. Let's see. So from the fire store, it'll go ahead and pick it up. And here you go. Here you get the answer clearly, right? See loading embedding models. This is there loading fire index and metadata. What is attention mechanism? All the information is over here. And this is the output that you are able to get. Right. Perfect. This this is what exactly uh I was actually talking about. But the best part is that we have created this in the form of a pipeline. You have data loader, you have embedding, you have vector store. Now for search what you can do is that you can integrate any LLMs over here. Right? So for this also I have written the code. Again I don't want to discuss it step by step line by line. So that it'll be again taking a lot amount of time to complete this. Right? So here I have my load_.env. You can just go ahead and load all these things. Groc API key is given over here. You can use it or you can use your own Groc API key. It's fine. Okay. And then we are doing the search, right? Wherein we are using this vector store do.query getting all the documents getting all the metadata and then we're giving some prompt and we are invoking it along with the LLM. So once we do this, it is superbly easy to execute this. Anyhow, you can do the research because I have discussed all these things in my Jupyter notebook, right? Uh now what I will do in my app.py py I'll see what changes needed to be added and uh what I will do is that I will first of all import rack search again from search dot search import rack search and then I will go ahead and initialize like this right and now I don't even require this okay now let's see whether it'll be able to give the summary or not it is loading from the vector store now I'm asking the question search and summarize This is the function here. What we do? We first of all do the query from the vector store that we were usually doing before. Then we give a prompt and then finally LLM will be able to give the output. So, so here you can see if my LLM is fine then I think I should be able to get an answer. So here you can see all the output is basically over here.

[2:07:44] So this was a complete idea, or a kind of crash course that I really wanted to give on the entire RAG. RAG is one of the most important use cases — that is what I always believe. Most of the companies are specifically building RAG applications. So I think this is

really really important and a super cool topic. I hope you like this particular video. This was it from my side. I'll see you on the next video. Thank you. Take care.